

EGYPT MULTI-ASSET AI PORTFOLIO SYSTEM

Team Guide & Plain-Language Overview

1. THE BIG PICTURE (READ THIS FIRST)

Imagine you are an investor in Egypt. You cannot only buy one stock or only keep cash. You face inflation, currency moves, high T-Bill rates, and sudden market crashes. A "normal" portfolio recipe from the US (like 60% stocks / 40% bonds) does not always work here.

Our project builds a smart assistant that answers one main question:

"How should we split money across Egyptian assets to get good returns without taking reckless risk — and when should we dial risk up or down?"

We do NOT promise a magic money machine. We build a clear, repeatable system that:

- Uses real Egyptian market data (indices, gold, T-Bills, real-estate fund)
- Cleans and understands that data properly
- Uses machine learning to read market mood (short term) and outlook (longer term)
- Uses portfolio math to suggest HOW MUCH to put in each asset
- Tests those ideas on history so we are honest about what worked and what did not
- Shows everything in notebooks, Power BI, and a web dashboard

Think of it as two brains working together:

- **Strategic Brain:** "Where should we be invested for the next few months?"
- **Tactical Brain:** "Should we be cautious or aggressive right now, today?"

The final answer combines both.

2. WHAT WE ARE TRYING TO ACHIEVE (GOALS)

- Build a full data pipeline from raw prices to model-ready datasets
- Understand each asset (risk, correlation, crashes, hedging role of gold, etc.)

- Forecast which assets might do better over roughly a quarter (63 trading days)
- Detect short-term buy / hold / sell style signals from patterns in the data
- Compute optimal portfolio weights (how much % in each asset)
- Stress-test with simulations and backtests
- Present results so anyone on the team can see and explain them

Success for us = a system that is understandable, reproducible, and defensible in front of professors or investors — not just "high accuracy on paper."

3. THE PROJECT, PART BY PART (SIMPLE)

A. Data (What we feed the system)

We use five main assets, all in Egyptian pounds:

- **EGX30:** Top 30 stocks (main equity benchmark)
- **EGX100:** Broader equity index (more companies)
- **Gold:** Priced in EGP (hedge when pound weakens / inflation rises)
- **T-Bills:** Short government debt (high yield "safe" anchor in Egypt)
- **Egyptians Real Estate Fund (EGREF):** Listed real-estate exposure

Data mostly comes from investing.com (daily prices since ~2015). We also cross-check with EGX and Yahoo Finance when needed.

Problem: Assets do not all start on the same date (e.g., gold history is shorter).

Solution: We align what we can and use statistical imputation (EM) so older history is not thrown away — but we always flag when data was "filled in."

B. Data Cleaning (Making numbers trustworthy)

Raw downloads are messy: commas in prices, % signs, missing volume, etc. We run a fixed cleaning pipeline (same steps for every asset):

```
load → fix column names → parse dates → sort → remove duplicates → convert prices
to numbers → build daily returns → save cleaned files
```

Special fixes:

- **T-Bills:** The raw file is NOT true investment returns; we rebuild returns from Central Bank yield schedules (otherwise bills look useless in the model).

- **Real-estate fund:** Prices move in jumps (appraisal style); we "unsmooth" so risk is not underestimated.
- **Extremes:** Extreme daily moves are capped (winsorized) so one weird day does not break the whole model.

C. Exploratory Data Analysis — EDA (Understanding the market)

Before modeling, we look at charts and statistics:

- How volatile is each asset?
- Do returns look "normal" or have fat tails (big surprises)?
- How correlated are assets? (e.g., does gold help when stocks fall?)
- Drawdowns: how bad can losses get?

We use Jupyter notebooks (one per asset + a general notebook) and a Power BI dashboard with three pages:

1. **Market overview:** Compare all assets
2. **Single-asset view:** Price, RSI, MACD
3. **Risk view:** Risk vs return scatter, return distributions

EDA is where we build intuition. It is not the final trading engine.

D. Feature Engineering (Signals the models can read)

Models do not see "price" alone. We create helper columns, for example:

- Past few days' returns (lags)
- RSI, MACD, Bollinger bands (common trading indicators)
- Distance from moving average
- Rolling volatility

These describe momentum, overbought/oversold conditions, and stress — the same ideas traders use, but computed automatically for ML.

E. Strategic Layer (Longer horizon — ~3 months)

Job: guess how each asset might perform over the next ~63 trading days (about one quarter, matching how often we rebalance strategically). Three regression-style models each make a forecast; we blend them:

- **Ridge Regression:** Simple, stable linear model
- **Random Forest:** Catches non-linear patterns

- **Support Vector (SVR):** Flexible boundaries

If models disagree or perform poorly on past tests, we shrink forecasts toward recent history so the optimizer does not bet on noise. Output: expected return per asset → goes into portfolio optimization.

F. Tactical Layer (Short horizon — days)

Job: classify near-term market action into: Sell signal (-1), Hold / neutral (0), Buy signal (+1).

Three classifiers vote: Logistic Regression, SVM (support vector machine), and XGBoost (gradient boosting trees). We also detect "regime" (e.g. bull trend, stress) and check momentum.

Important: Tactical accuracy is hard in finance. Our system often stays neutral and lowers position size when confidence is low — that is intentional, not a bug.

Output: signal + confidence + suggested exposure (volatility targeting).

G. Portfolio Optimization (How much in each asset)

Given expected returns and how assets move together (covariance matrix), we solve: *"What mix of assets gives the best risk-adjusted return?"*

Main result: the **Optimal portfolio** — maximum Sharpe ratio (best return per unit of risk), with rules:

- No short selling (weights ≥ 0)
- Weights sum to 100%
- Cap on total equity (EGX30 + EGX100) so we do not overload stocks

We also draw the efficient frontier (many possible portfolios from safe to aggressive) and run Monte Carlo simulations to see possible future outcomes and worst-case drawdowns.

H. Backtesting (Did this idea work in the past?)

We pretend to run the strategy historically:

- Train only on past data up to each rebalance date
- Pick weights, apply tactical overlay
- Move forward one quarter, repeat
- Compare to benchmarks (EGX30 alone, T-Bills, 60/40 mix)

Strict rule: never use future data when making a past decision (anti-leakage). Backtest answers: "Would this process have been reasonable?" — not a guarantee of future performance.

I. Outputs & Dashboards (What we show people)

- **intelligence_report.json:** One file with weights, signals, metrics, checks

- **Plots:** Allocations, frontier, backtest curves (generated by Python)
- **Power BI:** Interactive exploration for presentation
- **Web app (Next.js):** Reads the JSON and shows portfolio + tactical view

Folder map (simplified):

data/	→ cleaned CSV files
EDA/	→ exploration notebooks
src/	→ main Python engine
outputs/	→ JSON report + figures
Code/interactive-platform/	→ website

4. MODELS & OPTIMIZATION — MORE DETAIL

4.1 Why two layers?

Markets have two time scales:

- **Longer term (strategic):** Asset allocation — "own more gold or more equities?"
- **Shorter term (tactical):** Risk dial — "stay fully invested or pull back now?"

One model cannot do both well. Splitting keeps the design honest and easier to explain.

4.2 Strategic models (regression ensemble)

Target: future 63-day return for each asset. Each model learns from features (lags, RSI, MACD, etc.) on historical data.

- **Ridge:** Good baseline; penalizes overfitting
- **Random Forest:** Many decision trees vote; handles interactions
- **SVR:** Can fit curved relationships

Training discipline includes walk-forward testing and strict outlier limits. We use outputs carefully, with shrinkage and caps, because daily/weekly financial data is noisy.

4.3 Tactical models (classification ensemble)

Target: next-period direction bucket (sell / hold / buy) from return quantiles.

- **Logistic Regression:** Fast, interpretable probabilities
- **SVM:** Separates complex patterns in feature space
- **XGBoost:** Strong on tabular data; learns sequential corrections

Ensemble voting with penalties when models disagree, performance is unstable, probability calibration is poor, or classes are imbalanced.

4.4 How optimization uses model outputs

1. **Build inputs:** Expected return vector (from strategic forecasts + history blend), Covariance matrix (how assets move together), Risk-free rate (from T-Bill logic).
2. **Find weights:** Maximize Sharpe ratio subject to constraints (all weights positive, sum to 1, equity cap).
3. **Interpret result:** High T-Bill yields in Egypt push the optimizer toward assets that either pay enough risk premium (equities) or hedge macro stress (gold).
4. **Risk extras:** Efficient frontier, Monte Carlo simulations, and stress scenarios (equity crash, devaluation, rate spike shocks).

4.5 How strategic + tactical combine

The fusion layer compares strategic bias, tactical signal, and inflation regime. It produces a human-readable recommendation and confidence score. Volatility targeting ensures that if the market is swinging hard, the system suggests investing a fraction of full size — like driving slower in rain.

4.6 What to tell someone in one minute

"We built a pipeline for Egyptian multi-asset investing. We clean data, explore it, forecast quarterly returns with ML, classify short-term risk with another ML layer, then use portfolio optimization to suggest weights in EGX, gold, bills, and real estate. We backtest without cheating on future data, simulate risk, and show everything in reports and dashboards. Strategic = where to be; tactical = how much risk to take right now."

5. COMMON QUESTIONS (QUICK ANSWERS)

Q: Why is gold often a large weight?

A: In our framework it often hedges equity risk and inflation/currency stress when correlations and forecasts support that role.

Q: Why does tactical accuracy look low (~33%)?

A: Three-class near-random is ~33%. We use it as a risk dial with confidence, not as a guaranteed direction picker.

Q: What is EM imputation?

A: A statistical way to fill missing asset history so we can use long EGX data without throwing everything away when gold started late. Flagged in reports.

Q: Is this live trading?

A: No. It is research / decision support. Execution, fees, taxes, and liquidity are outside scope.

Q: Where is the "source of truth" after a run?

A: `outputs/intelligence_report.json` plus the technical report.

Everyone should read sections 1–3 of this file. Sections 4–5 are for members presenting the "how it works" part.